

Object-Oriented Programming

Alexis Shakas Ian Wasser (TA)

ETH Zurich

2026-05-12

Acknowledgements

These slides were prepared in collaboration with **Konstantinos Stavratis**, post TA in the course (2025).

Graduate from Department of Computer Science, ETH Zurich
Currently at **Arthrex**

Many thanks for the contribution.

Organizational

Lecture	Date	T.H.A.	Room
1: Kickoff	17/2	N	RZ D8
2: AI assistance and tools	24/2	N	RZ D8
3: Project-based teamwork	3/3	N	RZ D8
4: Intro to Git	10/3	N	RZ D8
5: Git continued	17/3	N	ML H41.1
6: Git collaboration	24/3	N	RZ D8
7: AI-assisted Python	31/3	Y	ML H41.1
8: The 3 R's in Python	14/4	Y	ML H41.1
9: The Python ecosystem	21/4	Y	RZ D8
10: cont. L9 & project feedback	28/4	N	RZ D8
11: Testing and debugging	5/5	Y	ML H41.1
12: OOP	12/5	Y	RZ D8
13: Project distribution	19/5	N	RZ D8
14: Project marketplace	26/5	N	RZ D8

T.H.A.: Take home assignment and peer review **today**.

1. **Theory** of Object-Oriented Programming
2. **Understand** the core OOP concepts
3. **Compare** OOP to procedural programming
4. **MiniTasks** — you'll write classes, inheritance, and polymorphism step by step

Take-home assignment 5 — your last one

This week you'll receive **Assignment 5** — the **final** take-home assignment. After this assignment comes in, I will inform you of your final assignment-related grade and you will have 1 week to review and tell me if you don't agree.

- Topic: **Rewriting** past assignments in an Object-Oriented manner
- Due: next lecture (19/5), peer review the lecture after
- Format: python/zipped folders on Moodle, plus a short self-reflection

What is OOP?

One common way to write code:

1. Break the problem into clear, ordered steps (including the inputs and outputs of each step)

One common way to write code:

1. Break the problem into clear, ordered steps (including the inputs and outputs of each step)
2. Treat each step as a task and write it as a function

One common way to write code:

1. Break the problem into clear, ordered steps (including the inputs and outputs of each step)
2. Treat each step as a task and write it as a function
3. Carefully call one function after another, starting from `main` to follow the planned sequenced steps. **You have defined an algorithm!**

One common way to write code:

1. Break the problem into clear, ordered steps (including the inputs and outputs of each step)
2. Treat each step as a task and write it as a function
3. Carefully call one function after another, starting from main to follow the planned sequenced steps. **You have defined an algorithm!**
4. Initialize and pass the data to work with

This way of writing code as a **sequence of functions** (focus) that operate **on data** (secondary) is called Procedural Programming.

Object Oriented Programming (OOP)

What is OOP?

Humans think of the world as **entities**, each with their own characteristics, which **interact** with one another.

Example: The **tall, old woman** **feeds** the **fluffy, brown dog**.

Woman

tall
old

— **feed** →

Dog

fluffy
brown

OOP attempts to imitate this exact way of thinking!

```
from human_behaviours import feed
# Initialize data
dog = {
    "species" : "Dog",
    "name" : "Hector",
    "age" : 6,
    "hunger" : 0,
}
cat = {
    "species" : "Cat",
    "name" : "Paw",
    "age" : 3,
    "hunger" : 30,
}
animals = [dog, cat]
man = {
    "species" : "Human",
    "name" : "Mary",
    "age" : 78,
    "hunger" : 0,
    "animal_food" : 100,
}
# Interaction(s)
feed(man, dog, 5)
```

```
from animals import Cat, Dog, Human

# Initialize data
dog = Dog("Hector", 6, 0)
cat = Cat("Paw", 3, 30)
animals = [dog, cat]

man = Human("Mary", 78, 0, 100)

# Interaction(s)
man.feed(dog, 5)
```

A **class** is the heart of **object-oriented programming** (OOP); it allows you to define a category of thing once, then create as many individual examples of it as you need, each carrying its own data and behavior.

A **class** is the heart of **object-oriented programming** (OOP); it allows you to define a category of thing once, then create as many individual examples of it as you need, each carrying its own data and behavior.

You create **instances** of a class by adding parentheses after its name:

```
from numpy.random import randn  
randn() # creates a random number
```

Those parentheses are the difference between **referring** to the class and **calling** the class.

Definition (Class)

A class is a conceptual **blueprint** that describes what all of its concrete instances **are**, and what they can **do**.

Example (Human Class)

What do *all humans* **do**?

1. eat
2. sleep
3. think
4. ...

What do *all humans* **have or are**?

1. age
2. hunger
3. height
4. ...

A class in Python — structure

A class definition has **three parts**:

```
class ClassName:                                # 1. header — keyword + name (PascalCase)

    def __init__(self, ...):                    # 2. constructor — sets up the state
        self.attribute = ...

    def some_method(self, ...):                 # 3. methods — what the object can do
        ...
```

- **Header** — keyword `class`, the name (PascalCase), then a colon
- **Constructor** `__init__` — runs automatically at instantiation
- **Methods** — regular functions whose first parameter is always `self`

The constructor — `__init__`

`__init__` runs **automatically** when an object is created. Its job is to set the **initial state** of that object.

```
class Dog:

    def __init__(self, name: str, age: int = 0, hunger: int = 0):
        self.name = name          # store the dog's name on this object
        self.age = age            # store the dog's age on this object
        self.hunger = hunger      # store the dog's hunger on this object
```

When we write `Dog("Hector", 5)`, Python:

1. creates a fresh empty Dog object,
2. passes it to `__init__` as `self`, then
3. runs the body — which fills in `self.name`, `self.age`, `self.hunger`.

Passing arguments at instantiation

When you write `Dog("Hector", 5)`:

```
class Dog:
    def __init__(self, name: str, age: int = 0, hunger: int = 0):
        ...

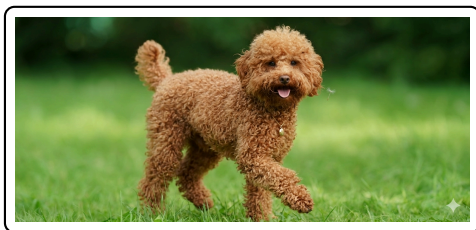
hector = Dog("Hector", 5)           # name="Hector", age=5, hunger=0 (default)
```

- `self` is **not** passed by you — Python fills it with the new object
- The other arguments fill `__init__`'s parameters **in order**
- Default values (e.g. `hunger: int = 0`) apply when you omit them
- The result of `Dog(...)` **is an object** assigned to the variable **hector**

Animal



Dog



Hector is an **instance** (**object**) of the concept (class) “Dog”.

Definition (Object)

An **object** is a **concrete instance** created from a class — it’s the actual “thing” you work with in your program.

The **object** has its **own specific values** for the attributes defined by the class.

self — “this particular object”

What is OOP?

`self` is **just a name** and by convention, the first parameter of every method. It refers to the **specific object** the method is called on; “Hector”, in the previous slide (except when using decorators).

self — “this particular object”

self is **just a name** and by convention, the first parameter of every method. It refers to the **specific object** the method is called on; “Hector”, in the previous slide (except when using decorators).

```
class Dog:
    def __init__(self, name: str):
        self.name = name          # attribute on THIS dog

    def bark(self):
        print(f"{self.name} says woof!")

d1 = Dog("Hector")
d2 = Dog("Rex")
d1.bark()                        # → "Hector says woof!"    self is d1
d2.bark()                        # → "Rex says woof!"       self is d2
```

Each object has its **own** self: d1.name and d2.name are independent.

MiniTask: Add Cat and Human

You have been given `animals.py` with a `Dog` class. **In the same file**, add the classes `Cat` and `Human` following the same style.

Cats

- have an age
- have a name
- can get hungry
- can eat an amount of food

Dogs

- have an age
- have a name
- can get hungry
- can eat an amount of food

Humans

- have an age, a name
- can get hungry
- carry an amount of food
- can eat an amount of food

Creating and using an object

To **instantiate** a class, call it like a function. `__init__` runs and you get back the new object.

```
from animals import Dog

hector = Dog("Hector", age=5, hunger=20)    # instantiate → __init__ runs

print(hector.name)                          # access an attribute → "Hector"
hector.eat(5)                               # call a method
print(hector.hunger)                        # → 15
```

You now have everything to **create** and **interact** with objects.

MiniTask: Feeding a hungry dog

Objects interact when a method takes another **object** as an argument.

1. Add `feed(animal, amount)` to **Human**: it calls `animal.eat(amount)` and reduces `self.animal_food` by `amount`.
2. Create a (hungry) Dog and a Human with enough food.
3. Call `mary.feed(hector, 5)` — passing the **dog object** into the **human's** method.
4. Print `hector.hunger` and `mary.animal_food` before and after.

Encapsulation

Definition

Encapsulation is the organization of **data** (state) together with **operations** that act on that data into a coherent unit (an **object**).



Class

Data and operations **bundled together** — not scattered.

Where is the logic that defines meaning?	
PP	Distributed across multiple functions and calling contexts
OOP	Localized inside the object that owns the data
Result	Meaning and rules are no longer implicit or scattered

MiniTask: Adding a rule — PP vs OOP

Suppose we want to enforce: **hunger can never go above 100.**

1. In `animals.py` (OOP) — which method do you change? Add the rule.
2. How would this be different in a procedural codebase? Which functions would you have to change?
3. Which of the 3R's does encapsulation enhance and why?

Discuss with your neighbour.

Where do you enforce “ <code>hunger ≤ 100</code> ”?	
OOP	Change the one method that raises hunger (e.g. the setter or wherever <code>self.hunger</code> is increased).
PP	Update every function/path that can increase hunger — the rule is not centralized.

```
# inside the Animal class (animals.py)
self.hunger = min(100, self.hunger + amount)
```

Which of the 3R's improves most?

- **Reusability** and **Reliability** — a single rule location removes duplication and inconsistent behavior.
- **Readability** also benefits: the rule lives next to the data it constrains.

Polymorphism

Definition (Polymorphism)

Polymorphism allows different objects to be used in the same way, even though they may behave differently internally.

- **What problem does it solve?** write code that works with a general concept, without knowing the concrete type in advance.

Definition (Polymorphism)

Polymorphism allows different objects to be used in the same way, even though they may behave differently internally.

- **What problem does it solve?** write code that works with a general concept, without knowing the concrete type in advance.
- **The core idea:** the same method call can do different things, depending on the **object** it is called on.

Definition (Polymorphism)

Polymorphism allows different objects to be used in the same way, even though they may behave differently internally.

- **What problem does it solve?** write code that works with a general concept, without knowing the concrete type in advance.
- **The core idea:** the same method call can do different things, depending on the **object** it is called on.
- **How does it work?** in Python, just define a method with the same name in another class.

New requirement: all animals must produce a sound — but each species produces a *different* one.

MiniTask: Make every animal greet

1. Add a `greet()` method to each of `Cat`, `Dog`, `Human`.
2. Create one object per class, put them in a list, and call `greet()` on each. What do you observe?

```
animals = [Cat(...), Dog(...), Human(...)]  
for animal in animals:  
    animal.greet()
```

Why is polymorphism useful?

The calling code knows nothing about the concrete classes:

```
for animal in animals:  
    animal.greet()           # works for Cat, Dog, Human, ...
```

Adding a new species:

Without polymorphism (PP)

- Update every `if-elif-else` chain
- Risk of forgetting one
- Calling code grows with each species

With polymorphism (OOP)

- Add one new class with `greet()`
- Calling code *doesn't change*
- Existing code keeps working

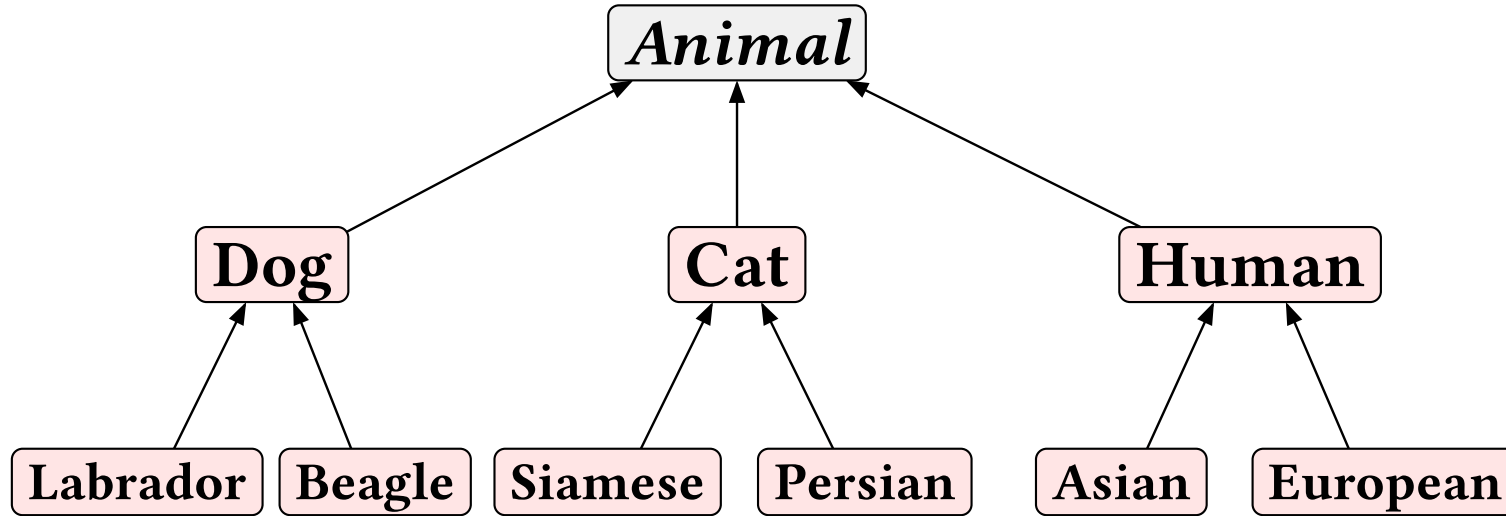
Inheritance

Definition (OOP)

A child class inherits the **attributes** and **methods** of all parent classes.

This means the child class:

- *Receives* code it did not write
- *Can use it* as it is
- *Can extend it* by adding new methods
- *Can change it* by overriding methods (later in the lecture)



Calling the parent constructor

When a subclass defines its own `__init__`, the parent's `__init__` does **not** run automatically. Call it explicitly with `super().__init__(...)`.

```
class Animal:
    def __init__(self, name: str, age: int = 0):
        self.name = name
        self.age = age

class Human(Animal):
    def __init__(self, name: str, age: int = 0, animal_food: int = 0):
        super().__init__(name, age)      # let the parent set name, age
        self.animal_food = animal_food   # then add what's specific to Human
```

`super()` refers to the parent class; `super().__init__(...)` runs its constructor on *this* object. Use it whenever the subclass needs **extra** attributes on top of the parent's

MiniTask: Extract the Animal class

In `animals.py`:

1. Identify identical **attributes** and **behaviours** between classes.
2. Add a new class `Animal` **at the top of the file**; move the common parts (including `__init__`) into it.
3. Make each child inherit: `class Dog:` → `class Dog(Animal):`.
Use `super().__init__(args)` inside the child's `__init__`.

Re-run your script; the output must be identical. Which one(s) of the 3Rs we have discussed does inheritance enhance and why?

Definition (Method Overriding)

Method overriding is a feature of inheritance where a subclass provides its own implementation of a method already defined in a superclass.

The overridden method:

- has the **same name**
- has the **same purpose**
- but provides **different behaviour** appropriate for the subclass

Method overriding — example

Animal defines a generic `greet()`. Dog **overrides** it with something specific.

```
class Animal:
    def __init__(self, name): self.name = name
    def greet(self):
        print(f"I am {self.name}.")

class Dog(Animal):
    def greet(self):                # same name as parent
        print(f"{self.name} says: Woof!")  # different behaviour

hector = Dog("Hector")
hector.greet()                    # → "Hector says: Woof!" ← Dog's version runs
```

Python looks up `greet` on `Dog` first, finds it, and uses it. The parent's version is **shadowed** — it still exists but is not called.

Use overriding when the subclass needs a **different implementation** of a method that the parent already defines.

Keep parent's behaviour

- Don't redefine the method.
- Inherit it as-is.

Specialize for the subclass

- Redefine the method with the same name.
- The subclass version takes over for instances of that class.

If the override needs the **parent's** logic too, call it explicitly with `super().method_name(args)` inside the override.

MiniTask: Age in human years

Express the age of each animal in human years.

1. Add a method `age_in_human_years()` to `Animal` returning `self.age` (default).
2. Override it in `Cat` and `Dog` with different ratios:
 - 1 human year = 15 cat years
 - 1 human year = 7 dog years
3. Call the method on a `Cat`, `Dog`, and a `Human` and print the results.

Assume age is stored in “human years”.

Information Hiding

Definition

Information hiding is the practice of deliberately restricting which parts of an object are *visible and accessible* from outside.

- Only a small, stable “*public*” *interface* is exposed; internal representation and implementation details stay inaccessible.
- This prevents external code from depending on internal details, so those details can change without affecting clients.
- Clients interact with objects in terms of *what they do*, not *how they are implemented*.

In Python, information hiding is **not enforced at the language level**. Instead, it is achieved through **naming conventions** that restrict *what should be visible and relied upon* by other code.

Rule of thumb:

- Names without underscores are part of the public interface
- Names starting with one underscore (_) are internal to the class and its subclasses
- Names starting with two underscores (__) are meant to be used only inside the defining class

MiniTask: Restrict the interface

1. Read through your classes in `animals.py`.
2. Using the rules of thumb (previous slide), make the interface (**member variables** and **methods**) as **restrictive** as you can.
3. From external code (e.g. `__main__`), attempt to access a member prefixed with:
 - one underscore (`_`)
 - two underscores (`__`)

What happens in each case?

Abstraction

Definition (Abstraction)

Abstraction in OOP is the practice of modeling only the relevant attributes and behaviours of a concept, while ignoring (complex) details that are not important.

- Abstraction focuses on *what* an object does, not *how* it does it.
- Abstraction is a **way of thinking and modeling**.
- Abstraction is the **design lens** used to decide encapsulation (what all behaviours of a concept are) and information hiding (how much of the behaviours are other objects allowed to see).

MiniTask: The polite animal

Extend `Human.feed()` so that after being fed, the `Animal`

1. greets the feeder, and
2. reports its age in human years.

Test it with a `Dog`, `Cat`, and another `Human`.

Abstraction — Reflection

Did you need to know:

- the *species* of the animal (Dog vs Cat vs ...)?
- *how* the age is computed?
- what fields the animal has (e.g. `_age`)?

What did you actually need?

- `An(y) Animal` *can* `greet()`
- `An(y) Animal` *can* `report age_in_human_years()`

That is abstraction.

Wrap-up

Recap

Today we covered:

- **Classes** — blueprint with structure, `__init__`, methods, `self`
- **Objects** — concrete instances, created by calling the class
- **Encapsulation** — bundling state + behaviour into one unit
- **Polymorphism** — same call, different behaviour per object
- **Inheritance** — *is-a* relationships, method overriding
- **Information hiding** — restricting the public interface
- **Abstraction** — model what matters, ignore what doesn't

You've also written your first multi-class file (`animals.py`) and refactored it through every OOP concept above.

Next two weeks

- **Next lecture (19/5):** Project distribution
- **26/5:** Project marketplace. You **need** to prepare a 10 minute presentation of your project for your classmates.
- **Take-home assignment 5:** due 19/5, peer review the lecture after — your **last** take-home assignment